Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЁТ

по лабораторной работе №7
по дисциплине «Логика и основы алгоритмизации в инженерных задачах»
на тему «Обход графа в глубину»

Выполнили ст. гр. 25BBB1:
Мингалеева А. Р.
Андреев М. А.

Приняли:
к.т.н., доцент Юрова О.В.
ст., преподаватель Деев М.В.

Пенза 2026

1. Цель работы

Изучить алгоритм обхода графа в глубину (поиск в глубину) и способы представления неориентированных графов в памяти компьютера (матрица смежности и списки смежности); реализовать на языке C++ генерацию матрицы смежности случайного графа, рекурсивный обход в глубину по матрице смежности, обход в глубину по спискам смежности и нерекурсивный обход в глубину с использованием стека.

2. Лабораторное задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа *G*. Выведите матрицу на экран.

2. Для сгенерированного графа осуществите процедуру обхода в глубину, реализованную в соответствии с приведенным выше описанием.

3.* Реализуйте процедуру обхода в глубину для графа, представленного списками смежности.

Лабораторное задание 2

1. Для матричной формы представления графов выполните преобразование рекурсивной реализации обхода графа к не рекурсивной.

3. Ход работы

3.1. Описание используемых алгоритмов

- Алгоритм генерации графа:

  Пользователь вводит количество вершин n. Граф задаётся квадратной матрицей смежности G размера n × n. Для каждой пары вершин i < j с помощью генератора случайных чисел (rand() % 2) выбирается 0 или 1, и это значение записывается в G[i][j] и в симметричную ей позицию G[j][i], поэтому матрица симметрична, что соответствует неориентированному графу. Диагональ остаётся нулевой, так как петли в графе отсутствуют. Начальное значение генератора берётся из текущего времени либо задаётся аргументом командной строки для

воспроизводимости результатов. Сгенерированная матрица выводится на экран; вершины нумеруются с единицы.

- Алгоритм обхода в глубину по матрице смежности (рекурсивный):
  Массив NUM из n логических значений показывает, посещена ли вершина; в начале все его элементы равны False (п. 1.1 алгоритма). Во внешнем цикле по v = 1…n для каждой ещё не посещённой вершины вызывается процедура DFS(v) (п. 1.2–1.3); этот цикл необходим для обхода графа, состоящего из нескольких компонент связности. Процедура DFS(v) помечает вершину v как посещённую (NUM[v] = True), выводит её номер на экран, затем просматривает v-ю строку матрицы по i = 1…n и для каждой вершины i, для которой G(v, i) = 1 и NUM[i] = False, рекурсивно вызывает DFS(i). Трудоёмкость обхода $O(n^2)$, так как для каждой вершины просматривается целая строка матрицы.

- Алгоритм преобразования матрицы смежности в списки смежности:
  Для каждой вершины i просматривается i-я строка матрицы. Номер каждого столбца j, в котором стоит единица, добавляется в список соседей вершины i. В результате получается массив из n списков, i-й из которых содержит номера всех вершин, смежных с вершиной i, в порядке возрастания. Для изолированной вершины список пуст.

- Алгоритм обхода в глубину по спискам смежности:
  Процедура устроена так же, как рекурсивная версия для матрицы, но вместо просмотра всей строки матрицы перебираются только элементы списка соседей adj[v]. Для каждого соседа u, который ещё не посещён, рекурсивно вызывается процедура обхода. Так как списки построены по возрастанию номеров вершин, порядок обхода совпадает с порядком обхода по матрице. Трудоёмкость обхода O(n + m), где m — число рёбер графа.

- Алгоритм нерекурсивного обхода в глубину:

  Рекурсивные вызовы заменены явным стеком (stack<pair<int, int>>). Элемент стека — пара (v, i): вершина и номер столбца, с которого нужно продолжить просмотр строки v. Это в точности то, что при рекурсии хранится в стеке вызовов (параметр v и счётчик цикла i). Для стартовой вершины выполняются действия п. 2.1–2.2 (пометка и вывод), и она помещается в стек с i = 0. Затем, пока стек не пуст, для вершины на его вершине ищется следующий столбец i, в котором G(v, i) = 1 и NUM[i] = False. Если такой столбец найден, счётчик в элементе стека увеличивается (после возврата просмотр продолжится со следующего столбца), найденная вершина помечается как посещённая, выводится на экран и помещается в стек (аналог вызова DFS(i)). Если строка просмотрена до конца, элемент извлекается из стека (аналог возврата из DFS(v)). Порядок посещения вершин получается таким же, как в рекурсивной версии.

- Особенности реализации:

  Матрица и списки смежности хранятся как vector<vector<int>>, метки посещения — как vector<bool>, стек — как stack<pair<int, int>>. Все три варианта обхода запускаются через одну общую функцию runTraversal, которая реализует алгоритм ПОГ (внешний цикл по непосещённым вершинам), а нужная процедура DFS передаётся ей параметром. Номера вершин при выводе увеличены на единицу. Файл с исходным кодом сохранён в кодировке UTF-8; для корректного вывода русского текста в консоли Windows перед запуском выполняется команда chcp 65001.

4. Результат работы программы

Для демонстрации программа запускалась для двух графов из 7 вершин; начальные значения генератора случайных чисел равны 3 и 5. После ввода числа вершин программа выводит матрицу смежности, списки смежности и

порядок обхода в глубину, полученный тремя способами: рекурсивно по матрице, по спискам смежности и без рекурсии.

```
Введите количество вершин графа: 7
Матрица смежности:
     1  2  3  4  5  6  7
  1  0  0  1  0  0  1  0
  2  0  0  0  0  1  0  0
  3  1  0  0  0  1  1  0
  4  0  0  0  0  1  0  1
  5  0  1  1  1  0  1  1
  6  1  0  1  0  1  0  1
  7  0  0  0  1  1  1  0

Списки смежности:
  1: 3 6
  2: 5
  3: 1 5 6
  4: 5 7
  5: 2 3 4 6 7
  6: 1 3 5 7
  7: 4 5 6

Обход в глубину (матрица, рекурсия):       1 3 5 2 4 7 6
Обход в глубину (списки смежности):        1 3 5 2 4 7 6
Обход в глубину (матрица, без рекурсии):   1 3 5 2 4 7 6
```

Рисунок 1 – Результат работы программы для связного графа

Для первого графа обход начался с вершины 1 и охватил все вершины одним вызовом DFS(1): из вершины 1 алгоритм перешёл в вершину 3, затем в 5, из неё — в 2 и, вернувшись, в 4, затем в 7 и 6. Результат обхода: 1 3 5 2 4 7 6.

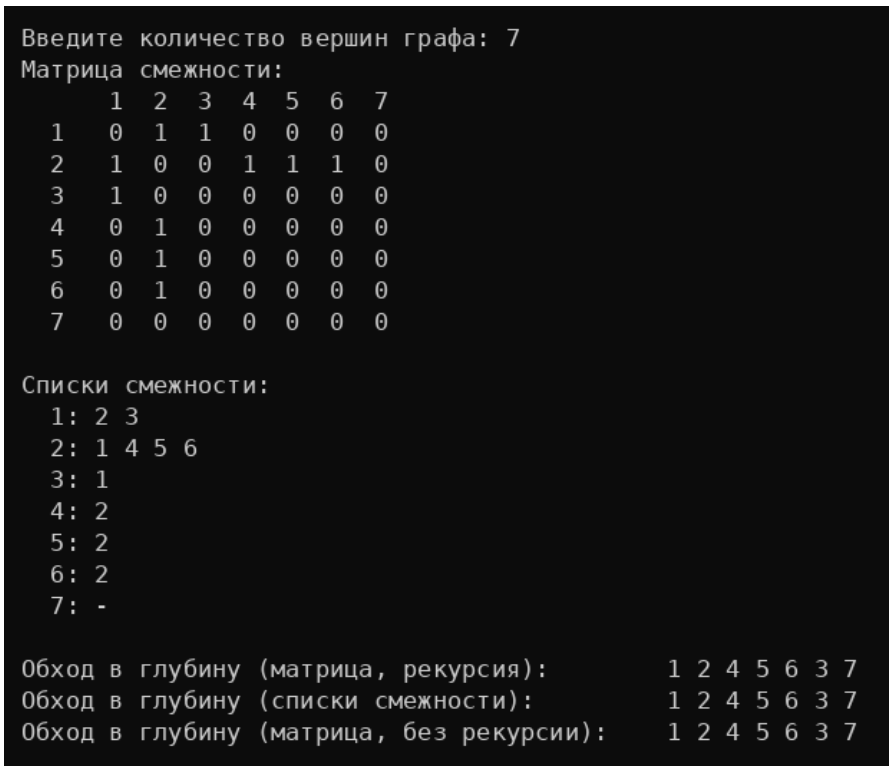


Рисунок 2 – Результат работы программы для несвязного графа

Второй граф несвязный: вершина 7 изолирована (её список смежности пуст). Вызов DFS(1) обходит компоненту связности {1, 2, 3, 4, 5, 6}, а вершина 7 посещается отдельным вызовом DFS(7) во внешнем цикле алгоритма ПОГ. Результат обхода: 1 2 4 5 6 3 7.

В обоих примерах порядок обхода, полученный рекурсивно по матрице смежности, по спискам смежности и без рекурсии, совпал. Дополнительно три реализации были сравнены на 300 случайных графах размером от 1 до 25 вершин — расхождений не обнаружено.

5. Вывод

В ходе лабораторной работы был изучен алгоритм обхода графа в глубину и реализован на языке C++ тремя способами: рекурсивно по матрице смежности, рекурсивно по спискам смежности и без рекурсии с использованием стека. Разработаны генерация случайной матрицы смежности неориентированного графа и построение списков смежности по матрице. Внешний цикл алгоритма обеспечивает обход всех компонент связности, что показано на примере графа с изолированной вершиной. Порядок обхода во

всех реализациях совпал, в том числе на 300 случайных графах, что подтверждает корректность преобразования рекурсии в итерацию: для этого достаточно хранить в стеке вершину и позицию просмотра её строки матрицы. Обход по спискам смежности эффективнее для разреженных графов ($O(n + m)$ против $O(n^2)$ для матрицы смежности).